



React Native

Formation 3 Jours — Jour 1

Modules 1 & 2 · Introduction + Bases React Native



Sommaire — Jour 1

01 Le dev natif & la fragmentation

02 Solutions cross-platform

03 Positionnement React Native

04 La New Architecture (2024-2025)

05 Rappels ES6 / ES2015+

06 Transpileurs : Babel & Hermes

07 React, JSX & Hooks

08 React Native — Principes clés

09 Installation & Configuration

10 Outils de développement & debug

11 TP — Installation & debugger

Module 1 · Introduction

Le développement mobile, la fragmentation et les solutions cross-platform

Le développement natif

- **iOS** : Swift / Objective-C — Xcode — App Store
- **Android** : Kotlin / Java — Android Studio — Play Store
- **Avantage** : performances maximales, accès complet aux API natives, UX 100% native
- **Inconvénient majeur** : deux codebases à maintenir en parallèle
- **Coût** : équipes doubles, releases décalées, divergence UX

Un projet natif pur nécessite deux équipes distinctes pour cibler les deux plateformes majeures.



Hétérogénéité & fragmentation

- **iOS** : fragmentation quasi nulle — Apple contrôle matériel & logiciel
- **Android** : +24 000 modèles d'appareils actifs, Android 8 à 15 en production
- **Résolutions** : de 320px (anciens) à 1440px (haut de gamme)
- **Surcouches constructeurs** : Samsung One UI, Xiaomi MIUI, Oppo ColorOS...
- **Conséquence** : comportements différents selon l'appareil et la version OS



Chiffres marché 2026

Plateforme	Part de marché mondiale
Android	~72 %
iOS	~27 %
Autres	~1 %

- **Objectif** : couvrir les deux plateformes avec un budget maîtrisé
- **Solution** : le développement cross-platform — partager le maximum de code



Panorama des solutions cross-platform

- **WebView hybride** : Cordova, Capacitor, Ionic — HTML/CSS/JS dans une vue web
- **Compilé Flutter** : Dart + moteur Impeller — dessine chaque pixel nativement
- **Composants natifs JS** : React Native — pont JS vers composants natifs réels
- **Partage de logique** : Kotlin Multiplatform — code partagé, UI native séparée
- **Multi-target .NET** : MAUI — C# vers iOS/Android/Windows/macOS

Comparatif 2026

React Native

- Langage : JavaScript / TypeScript
- Rendu : composants natifs réels
- Performance : $\approx$ natif (New Arch)
- Écosystème : npm immense
- Courbe : facile si React connu
- Adoption : Meta, Shopify, Microsoft

Flutter

- Langage : Dart (propre à Google)
- Rendu : moteur Impeller
- Performance : $\approx$ natif
- Écosystème : pub.dev
- Courbe : Dart à apprendre
- Adoption : Google, BMW, Alibaba



Historique React Native

- Jan. 2015 : annoncé à la React.js Conf par Meta
- Mar. 2015 : open source — iOS uniquement
- Sep. 2015 : support Android ajouté
- 2018 : annonce de la New Architecture (JSI, Fabric, TurboModules)
- Oct. 2024 : RN 0.76 — New Architecture activée par défaut
- Fév. 2025 : RN 0.78 — React 19 intégré
- Juin 2025 : architecture legacy officiellement gelée
- 2025 : 80 % d'adoption de la New Architecture dans la communauté



La philosophie React Native

"Learn Once, Write Anywhere" — pas "Write Once, Run Everywhere"

- Chaque plateforme garde son look & feel natif authentique
- React Native utilise les vrais composants iOS et Android (pas une simulation)
- Les développeurs React web peuvent créer des apps mobiles rapidement
- On écrit du JS/TypeScript — pas du Swift, pas du Kotlin



Qui utilise React Native en 2026 ?

- **Meta** : Facebook, Instagram, Messenger
- **Microsoft** : Outlook Mobile, Xbox, Teams
- **Shopify** : app mobile principale (témoignage de migration réussi)
- **Coinbase, Discord, Wix, Bloomberg, Walmart**
- **Airbnb** avait abandonné en 2018 — leurs arguments sont largement résolus par la New Architecture



La New Architecture — Vue d'ensemble

L'ancienne architecture reposait sur un **bridge asynchrone JSON** :

```
JS Thread ← JSON Bridge (async, coûteux) → Native Thread
```

Trois nouveaux composants remplacent ce bridge :

- **JSI** (JavaScript Interface) : communication synchrone JS ↔ C++ natif
- **Fabric** : nouveau renderer — rendu synchrone, multi-thread
- **TurboModules** : chargement lazy des modules natifs

⚡ JSI — JavaScript Interface

- Remplace le bridge par des appels synchrones directs en C++
- JavaScript peut tenir des références directes aux objets natifs
- Zéro sérialisation JSON → latence quasi nulle
- Ouvre la porte aux intégrations avancées (Reanimated 3, Vision Camera, Nitro)

```
// JSI : JavaScript appelle directement du code natif
const module = global.__turboModuleProxy('DeviceInfo');
const name = module.getDeviceName(); // synchrone !
```



Fabric & TurboModules

Fabric (nouveau renderer UI) :

- Rendu synchrone — le thread JS lit le layout immédiatement
- Support complet de React Concurrent (Suspense, transitions)
- Multi-threading — priorités de rendu, animations fluides à 60fps

TurboModules :

- Chargement **lazy** — les modules natifs ne s'initialisent qu'à leur premier usage
- Typage fort : interfaces générées via **CodeGen** (TypeScript → C++)
- Réduit le temps de démarrage de l'app significativement



Gains mesurés — New Architecture

Métrique	Ancienne	New Architecture
Temps de démarrage	2–4 s	< 1 s (–40 %)
Framerate scroll complexe	45–55 fps	60 fps stables
Consommation mémoire	base 100 %	–20 à 30 %
Latence JS  natif	5–15 ms	< 1 ms

*RN 0.82+ : New Architecture **toujours activée**, impossible de désactiver.*

Module 2 · React Native

ES6+, JSX, React, Installation & Outils de développement

ES6 — Pourquoi c'est crucial

- React Native s'écrit en **JavaScript moderne** (ES6 à ES2022+)
- **TypeScript** est le standard de fait en 2025
- **Babel** transpile le code vers du JS compatible
- La maîtrise d'ES6+ est **prérequis** pour lire et écrire du React Native



Arrow Functions & Destructuration

```
// Arrow function
const ajouter = (a, b) => a + b;
const saluer = nom => `Bonjour ${nom} !`;

// Destructuration objet
const { nom, age, ville = 'Paris' } = user;

// Destructuration tableau
const [premier, ...reste] = ['a', 'b', 'c'];

// Dans les paramètres
const afficher = ({ nom, age }) => `${nom} a ${age} ans`;

// Spread & copie immuable
const userMaj = { ...user, age: user.age + 1 };
```

Modules, Async/Await & Optional Chaining

```
// Modules ES6
export const PI = 3.14;
export default function Calculatrice() {}
import Calculatrice, { PI } from './math';

// Async / Await
async function charger(id) {
  try {
    const res = await fetch(`/api/users/${id}`);
    return await res.json();
  } catch (err) {
    console.error(err);
  }
}

// ES2020+ – Optional Chaining & Nullish Coalescing
const ville = user?.adresse?.ville ?? 'Inconnue';
const dernier = tableau.at(-1);
```

◆ TypeScript dans React Native

- TypeScript est le **standard de facto** en 2025 — template Expo SDK 55 par défaut
- Détecte les erreurs à la compilation, pas à l'exécution

```
// Typage des props et du state
interface CarteProps {
  titre: string;
  note: number;
  onPress: (id: string) => void;
  style?: ViewStyle; // Optionnel
}

const Carte: React.FC<CarteProps> = ({ titre, note, onPress }) => (
  <TouchableOpacity onPress={() => onPress(titre)}>
    <Text>{titre} – ★ {note}</Text>
  </TouchableOpacity>
);
```

⚡ Hermes — Le moteur JS de React Native

- Créé par Meta — open source depuis 2019
- Actif par défaut depuis React Native 0.70
- Compilation AOT (ahead-of-time) → bytecode pré-compilé
- Avantages :
 - Démarrage plus rapide
 - Moins de consommation mémoire
 - Source maps précises pour le débogage
- En 2025 : supporte ES2022 complet + features React 19



React — Les fondamentaux

```
// Composant fonctionnel (le seul style recommandé en 2025)
const MonComposant: React.FC<{ nom: string }> = ({ nom }) => {
  const [count, setCount] = useState(0);

  // useEffect = lifecycle (mount, update, unmount)
  useEffect(() => {
    console.log(`Monté - ${nom}`);
    return () => console.log('Démonté');
  }, [nom]);

  const incremener = useCallback(() => setCount(c => c + 1), []);

  return (
    <View>
      <Text>Bonjour {nom} - {count}</Text>
      <Button title="+" onPress={incremener} />
    </View>
  );
};
```



JSX — Règles essentielles

```
// Un seul élément racine (ou Fragment <>)  
return (  
  <>  
    <Text>Ligne 1</Text>  
    <Text>Ligne 2</Text>  
  </>  
)  
;  
  
// Expressions JS entre accolades  
const score = 42;  
<Text>Score : {score > 40 ? '🔥 Excellent' : 'Bien'}</Text>  
  
// Attributs en camelCase  
<View backgroundColor="blue" onPress={handler} />  
  
// Composants auto-fermants si pas d'enfants  
<Image source={logo} />  
  
// ⚠️ Tout texte doit être dans <Text>  
<Text>Jamais de texte nu dans JSX</Text>
```

React Native vs React Web

Élément Web	Équivalent React Native
<code><div></code>	<code><View></code>
<code><p></code> , <code></code>	<code><Text></code>
<code></code>	<code><Image></code>
<code><input></code>	<code><TextInput></code>
<code><button></code>	<code><TouchableOpacity></code> ou <code><Pressable></code>
CSS standard	<code>StyleSheet.create()</code>
<code>onClick</code>	<code>onPress</code>
React Router	Expo Router / React Navigation



Thread model React Native

JS Thread (Hermes) Logique métier, state React, hooks
UI Thread (Main Thread) Rendu Fabric, animations natives
Background Threads Réseau, I/O, modules natifs

New Architecture (JSI) : communication
SYNCHRONE entre JS Thread et C++ natif



Installation — Prérequis système

Commun iOS + Android :

- Node.js $\geq$ 18.x LTS (22.x recommandé)
- Watchman (macOS/Linux)
- Git $\geq$ 2.x

Pour Android :

- Android Studio + SDK Android 14+ (API 34+)
- JDK 17 (OpenJDK)
- Variables `ANDROID_HOME` et `PATH` configurées

Pour iOS (macOS uniquement) :

- Xcode $\geq$ 15.x
- CocoaPods $\geq$ 1.12



Expo vs React Native CLI — 2025

Critère	Expo SDK 55	React Native CLI
Mise en route	~2 min	30–60 min
New Architecture	✓ par défaut	✓ configurable
Build cloud	✓ EAS Build	✗ local
Modules natifs custom	Via plugins Expo	Accès complet
Recommandé pour	90 % des projets	Code natif avancé

En 2025 : commencer avec Expo pour la grande majorité des projets.

Créer un projet Expo

```
# Créer un projet (New Architecture activée par défaut – SDK 55)  
npx create-expo-app@latest MonApp
```

```
cd MonApp
```

```
# Lancer le serveur de développement Metro  
npx expo start
```

```
# Raccourcis dans le terminal :  
# i → Simulateur iOS (macOS uniquement)  
# a → Émulateur Android  
# w → Navigateur web  
# Scan QR → Expo Go sur appareil physique
```

Structure d'un projet Expo SDK 55

```
MonApp/
├── app/                                # Routes Expo Router (file-based)
│   ├── _layout.tsx                    # Layout racine
│   ├── index.tsx                      # Route "/" – écran principal
│   └── (tabs)/                         # Groupe routes avec onglets
│       ├── _layout.tsx
│       ├── index.tsx
│       └── explore.tsx
├── components/                        # Composants réutilisables
├── assets/                            # Images, fonts, icônes
├── constants/                         # Couleurs, config globale
├── hooks/                             # Custom hooks
├── app.json                           # Configuration Expo
└── tsconfig.json                      # Configuration TypeScript
```

Créer un projet React Native CLI

```
# Initialiser
npx @react-native-community/cli@latest init MonApp

# Structure native
MonApp/
├── android/      # Projet Gradle natif Android
├── ios/          # Projet Xcode natif iOS
├── src/          # Code JS/TS
├── App.tsx       # Point d'entrée
└── metro.config.js # Config bundler

# Lancer
npx react-native run-android
npx react-native run-ios

# Configurer Android – ~/.zshrc ou ~/.bashrc
export ANDROID_HOME=$HOME/Library/Android/sdk
export PATH=$PATH:$ANDROID_HOME/platform-tools
```



Fast Refresh & Debug Menu

```
# Fast Refresh – rechargement instantané sans perte de state
# Activé par défaut dans Metro

# Raccourcis clavier sur émulateur/simulateur :
# Simulateur iOS : Cmd+D → Menu développeur
#                  Cmd+R → Recharger
# Émulateur Android : Cmd+M (Mac) / Ctrl+M (Win) → Menu dev
#                  R R → Double R pour recharger

# Ouvrir React Native DevTools
# → Taper 'j' dans le terminal Metro
# → Ou sélectionner "Open DevTools" dans le menu dev
```

React Native DevTools (nouveau 2025)

- Remplace progressivement Flipper depuis 2025
- Ouvrir : touche **j** dans Metro ou menu développeur
- React Components : inspecter l'arbre de composants
- Performance Profiler : identifier les re-renders coûteux
- Network Inspector : toutes les requêtes HTTP/WebSocket
- Console : logs centralisés, filtrage, recherche



Expo Go vs Development Build

```
# Expo Go : tester rapidement – SANS build natif
# → Limité aux APIs incluses dans le SDK Expo (suffisant pour débiter)

# Development Build : version dev personnalisée
# → Accès à TOUTES les bibliothèques natives

# Build local (requiert Xcode / Android Studio)
npx expo run:ios      # iOS
npx expo run:android  # Android

# Build cloud EAS (sans macOS pour iOS !)
npx eas build --profile development --platform ios
npx eas build --profile development --platform android
```



Extensions VS Code recommandées

- **React Native Tools** (Microsoft) — debug et configurations de lancement
- **ESLint** — linting en temps réel avec règles React Native
- **Prettier** — formatage automatique du code
- **TypeScript Nightly** — support TypeScript amélioré
- **Auto Rename Tag** — synchronisation des balises JSX ouvrante/fermante
- **Error Lens** — affiche les erreurs inline dans le code

TP — Jour 1

Objectif : Installer React Native et maîtriser le debugger



TP — Étapes

Étape 1 — Environnement

```
node --version # Vérifier >= 18.x
npx create-expo-app@latest ReactNativeTP
cd ReactNativeTP && npx expo start
```

Étape 2 — Premier composant

Modifier `app/index.tsx` : afficher un compteur avec `useState`

Étape 3 — Debug

- Ouvrir React Native DevTools (`j` dans Metro)
- Inspecter l'arbre de composants
- Ajouter un `console.log` et l'observer dans la console

Étape 4 — Exploration

Lire la structure de `app.json` et `tsconfig.json`



TP — Code du composant compteur

```
// app/index.tsx
import { useState, useCallback } from 'react';
import { View, Text, TouchableOpacity, StyleSheet } from 'react-native';

export default function HomeScreen() {
  const [count, setCount] = useState(0);
  const inc = useCallback(() => setCount(c => c + 1), []);
  const dec = useCallback(() => setCount(c => Math.max(0, c - 1)), []);
  const reset = useCallback(() => setCount(0), []);

  return (
    <View style={s.container}>
      <Text style={s.titre}>Mon Premier Composant</Text>
      <Text style={s.count}>{count}</Text>
      <View style={s.row}>
        <TouchableOpacity style={s.btn} onPress={dec}>
          <Text style={s.btnTxt}>-</Text>
        </TouchableOpacity>
        <TouchableOpacity style={[s.btn, s.btnReset]} onPress={reset}>
          <Text style={s.btnTxt}>Reset</Text>
        </TouchableOpacity>
        <TouchableOpacity style={s.btn} onPress={inc}>
          <Text style={s.btnTxt}>+</Text>
        </TouchableOpacity>
      </View>
    </View>
  );
}
```



TP — StyleSheet

```
const s = StyleSheet.create({
  container: {
    flex: 1,
    alignItems: 'center',
    justifyContent: 'center',
    backgroundColor: '#f5f7fa',
  },
  titre: {
    fontSize: 20,
    fontWeight: '600',
    color: '#0d47a1',
    marginBottom: 32,
  },
  count: {
    fontSize: 80,
    fontWeight: '800',
    color: '#0d47a1',
    marginBottom: 32,
  },
  row: { flexDirection: 'row', gap: 12 },
  btn: { backgroundColor: '#0d47a1', borderRadius: 10, paddingHorizontal: 28, paddingVertical: 14 },
  btnReset: { backgroundColor: '#e65100' },
  btnTxt: { color: '#fff', fontSize: 18, fontWeight: '700' },
});
```

? Questions de révision — Jour 1

- Quels sont les 3 composants de la New Architecture React Native ?
- Quelle est la différence entre JSI et l'ancien bridge JSON ?
- En quoi "Learn Once, Write Anywhere" diffère-t-il de "Write Once, Run Anywhere" ?
- Qu'est-ce qu'Hermes et pourquoi est-il utilisé ?
- Quelle est la différence entre Expo Go et un Development Build ?
- Quand préférer Expo SDK plutôt que React Native CLI ?

